AI Flashcard Generator

Project source code PDF

A simple Flask + SQLite web app. Enter a topic to generate study flashcards and reveal each answer.

Project structure

```
AI_Flashcard_Generator/\n  app.py\n  templates/index.html\n  static/style.css\n  static/script.js\n  requirements.txt
```

How to run

```
1. Install Python 3\n2. Open a terminal in the project folder\n3. Run: pip install -r requirements.txt\n4. Run: python app.py\n
```

requirements.txt

```
Flask>=3.0,<4.0
```

app.py

```
from flask import Flask, render_template, request, jsonify
import sqlite3
from pathlib import Path

app = Flask(__name__)
DB_PATH = Path(__file__).with_name("flashcards.db")

QUESTION_BANK = {
    "python": [
        ("What is Python?", "Python is a high-level, interpreted programming language."),
        ("What is a list?", "A list is an ordered, mutable collection."),
        ("What does len() do?", "It returns the number of items in an object.")
    ],
    "html": [
        ("What does HTML stand for?", "HyperText Markup Language."),
        ("What is an HTML element?", "An element is a page building block."),
        ("Which tag creates a hyperlink?", "The &lt;a&gt; tag creates a hyperlink.")
    ],
    "java": [
        ("What is Java?", "Java is a high-level, class-based programming language."),
        ("What is a class?", "A class is a blueprint for creating objects."),
        ("What does JVM stand for?", "Java Virtual Machine.")
    ],
    "general": [
        ("What is an algorithm?", "A finite sequence of steps to solve a problem."),
        ("What is a database?", "An organized collection of data."),
        ("What is a variable?", "A named storage location for a value.")
    ]
}

def init_db():
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS flashcards (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                topic TEXT NOT NULL,
                question TEXT NOT NULL,
                answer TEXT NOT NULL
            )
        """)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json(silent=True) or {}
    topic = (data.get("topic") or "general").strip()
    if not topic:
        return jsonify({"error": "Enter a topic."}), 400
    cards = QUESTION_BANK.get(topic.lower(), QUESTION_BANK["general"])
    with sqlite3.connect(DB_PATH) as conn:
        conn.executemany(
            "INSERT INTO flashcards (topic, question, answer) VALUES (?, ?, ?)",
            [(topic, q, a) for q, a in cards]
        )
    return jsonify({"topic": topic, "cards": [
        {"question": q, "answer": a} for q, a in cards
    ]})

@app.route("/history")
def history():
    with sqlite3.connect(DB_PATH) as conn:
        conn.row_factory = sqlite3.Row
        rows = conn.execute(
            "SELECT topic, question, answer FROM flashcards ORDER BY id DESC LIMIT 100"
        ).fetchall()
    return jsonify([dict(row) for row in rows])

if __name__ == "__main__":
    init_db()
    app.run(debug=True)
```

templates/index.html

```
&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;
&lt;head&gt;
  &lt;meta charset="UTF-8"&gt;
  &lt;meta name="viewport" content="width=device-width, initial-scale=1.0"&gt;
  &lt;title&gt;AI Flashcard Generator&lt;/title&gt;
  &lt;link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}"&gt;
&lt;/head&gt;
&lt;body&gt;
  &lt;main class="container"&gt;
    &lt;h1&gt;AI Flashcard Generator&lt;/h1&gt;
    &lt;p class="subtitle"&gt;Enter a topic and create quick study flashcards.&lt;/p&gt;
    &lt;form id="topicForm"&gt;
      &lt;label for="topic"&gt;Study topic&lt;/label&gt;
      &lt;input id="topic" name="topic" placeholder="Example: Python" required&gt;
      &lt;button type="submit"&gt;Generate Flashcards&lt;/button&gt;
    &lt;/form&gt;
    &lt;p id="status" role="status"&gt;&lt;/p&gt;
    &lt;section id="cards" class="cards"&gt;&lt;/section&gt;
  &lt;/main&gt;
  &lt;script src="{{ url_for('static', filename='script.js') }}"&gt;&lt;/script&gt;
&lt;/body&gt;
&lt;/html&gt;
```

static/style.css

```
* { box-sizing: border-box; }
body {
  margin: 0; min-height: 100vh; padding: 30px 16px;
  font-family: Arial, sans-serif;
  background: linear-gradient(135deg, #d9f5ff, #ffd6f6, #fff1b8);
  color: #30243a;
}
.container {
  max-width: 760px; margin: 30px auto; padding: 30px;
  background: white; border-radius: 20px;
  box-shadow: 0 8px 28px rgba(70, 40, 90, .12);
}
h1 { text-align: center; color: #d63384; margin-top: 0; }
.subtitle { text-align: center; color: #66566d; }
form { display: grid; gap: 12px; margin: 26px 0; }
label { font-weight: bold; }
input {
  padding: 13px; border: 1px solid #d7c9df;
  border-radius: 10px; font-size: 16px;
}
button {
  padding: 13px; border: 0; border-radius: 10px;
  background: #8b45c6; color: white; font-size: 16px;
  font-weight: bold; cursor: pointer;
}
button:hover { background: #7135a5; }
.cards { display: grid; gap: 14px; }
.card {
  padding: 18px; border: 1px solid #eadcf1;
  border-radius: 14px; background: #fffaff;
}
.card h3 { margin: 0 0 10px; color: #7135a5; }
.answer { display: none; margin-top: 10px; line-height: 1.5; }
.card.revealed .answer { display: block; }
.card .toggle { margin-top: 12px; padding: 9px 12px; font-size: 14px; }
#status { text-align: center; color: #7135a5; }
```

static/script.js

```
const form = document.getElementById("topicForm");
const cardsArea = document.getElementById("cards");
const statusText = document.getElementById("status");

form.addEventListener("submit", async (event) =&gt; {
  event.preventDefault();
  const topic = document.getElementById("topic").value.trim();
  cardsArea.innerHTML = "";
  statusText.textContent = "Creating your flashcards...";

  try {
    const response = await fetch("/generate", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ topic })
    });
    const data = await response.json();
    if (!response.ok) throw new Error(data.error || "Could not create cards.");

    statusText.textContent = `Created ${data.cards.length} cards for ${data.topic}.`;
    data.cards.forEach((item, index) =&gt; {
      const card = document.createElement("article");
      card.className = "card";
      const question = document.createElement("h3");
      question.textContent = `${index + 1}. ${item.question}`;
      const answer = document.createElement("p");
      answer.className = "answer";
      answer.textContent = item.answer;
      const button = document.createElement("button");
      button.className = "toggle";
      button.type = "button";
      button.textContent = "Show Answer";
      button.addEventListener("click", () =&gt; {
        card.classList.toggle("revealed");
        button.textContent = card.classList.contains("revealed")
          ? "Hide Answer" : "Show Answer";
      });
      card.append(question, answer, button);
      cardsArea.appendChild(card);
    });
  } catch (error) {
    statusText.textContent = error.message;
  }
});
```